



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Informe De Implementación

Análisis Sintáctico (Preguntas 4 y 5)

Profesor:

Márquez Félix

Alumnos:

Aguilera Rhixeidys V-30.851.503

Albornoz Ronniel V- 24.889.173

Bravo Yvanna V- 28.530.717

Curbata Emilio V- 28.214.578

Ciudad Guayana, julio de 2026

Introducción

Este informe documenta la implementación de los dos desafíos de programación del Tema 5: (4) un lexer/parser para la sección networks de archivos docker-compose.yml, replicado en tres lenguajes distintos (C, Java y Python) para un experimento de carga; y (5) un asistente de programación híbrido que combina técnicas tradicionales de compiladores (autómatas/regex, parser recursivo descendente) con una capa de apoyo tipo IA (fallback por umbral de confianza y sugerencias ante errores) para un subconjunto de Python llamado UnegScript. Ambos programas se apoyan en los conceptos teóricos desarrollados en el Informe de Investigación: construcción de AST (Pregunta 1), estrategias descendentes/ascendente (Pregunta 2) y técnicas de manejo de errores usadas por los generadores de parsers (Pregunta 3).

1. Descripción Del Trabajo Realizado

Pregunta 4. Se definió un subconjunto de YAML (indentación fija de 2 espacios) que cubre las claves relevantes de docker-compose.yml para describir las interfaces de red del sistema: services.<nombre>.networks (lista de redes a las que se conecta cada servicio) y networks.<nombre> con sus propiedades driver, external, attachable, driver_opts e ipam.config (subredes/gateway). Se generaron 10 archivos de prueba de tamaño creciente (2 a 20 servicios, 2 a 10 redes; gen_samples.py), cumpliendo el requisito $5 < n < 20$. Se implementó el mismo algoritmo (lexer por líneas sensible a indentación + parser recursivo descendente dirigido por nivel, análogo a INDENT/DEDENT de Python) en Python, Java y C, y se construyó un arnés (run_experiment.py) que primero valida que los tres parsers produzcan resultados idénticos y luego mide el tiempo de ejecución de cada uno sobre los 10 archivos (5 repeticiones por combinación archivo/lenguaje), generando CSV y gráficas.

Pregunta 5. Se implementó uneg_assistant.py: un lexer que tokeniza con expresiones regulares y, cuando un identificador no coincide exactamente con una palabra reservada, calcula la similitud (ratio de difflib.SequenceMatcher, equivalente a la fórmula de Levenshtein descrita en la lectura complementaria del tema) contra las palabras clave conocidas. Si la confianza supera 0.8 se corrige automáticamente; si la confianza está en zona ambigua se consulta a un módulo de IA simulado (ai_suggest), que emula la respuesta de un LLM. El parser es recursivo descendente con un token de lookahead; cuando encuentra un error sintáctico, aplica recuperación en modo pánico (descarta tokens hasta un punto de sincronización: ';', 'else' o fin de archivo) y, antes de descartar, consulta al módulo de IA para ofrecer una sugerencia legible sobre la causa probable del error. Se probó exactamente con el ejemplo de código con errores del enunciado.

2. Explicación de cada programa

2.1 pregunta4_docker_parser/gen_samples.py

Genera los 10 archivos docker-compose-01.yml ... docker-compose-10.yml con número creciente de servicios y redes, siempre con indentación de 2 espacios. Es el único punto donde se controla el "tamaño de la carga" del experimento.

2.2 pregunta4_docker_parser/python/lexer_parser.py

- **Lexer (tokenize_lines):** convierte cada línea no vacía en un LineToken {nivel, es_lista, clave, valor}, calculando el nivel de indentación (equivalente conceptual a los tokens INDENT/DEDENT).
- **Parser (Parser.parse_block):** recursión dirigida por nivel: mientras el siguiente token tenga el mismo nivel se agrega al nodo actual; si tiene valor se guarda como par clave-valor, si no, se interpreta como bloque anidado y se llama recursivamente con nivel+1. Las líneas de lista (- item) se acumulan en un arreglo.
- **extract_networks_summary:** recorre el árbol genérico resultante y arma el resumen estructurado (nombre de red, driver, external, attachable, subredes) usado para validar consistencia y para la salida --json.

2.3 pregunta4_docker_parser/java/LexerParser.java

Réplica exacta del algoritmo anterior usando LinkedHashMap/List como estructura genérica del árbol. Se ejecuta con el lanzador de código fuente de Java 11+ (java LexerParser.java archivo.yml), sin requerir javac por separado, lo cual es relevante para la discusión de resultados (ver sección e).

2.4 pregunta4_docker_parser/c/lexer_parser.c

Réplica del mismo algoritmo, pero sin árbol genérico dinámico: el parser reconoce directamente las claves de interés (services, networks, driver, ipam, config, subnet, ...) usando

arreglos de tamaño fijo, aprovechando que C no tiene contenedores genéricos ni recolector de basura incorporados. Se compila una sola vez (gcc -O2) y se ejecuta como binario nativo.

2.5 pregunta4_docker_parser/run_experiment.py

Arnés del experimento: 1) ejecuta los tres programas con --json sobre cada archivo y compara las salidas para garantizar que miden lo mismo; 2) repite cada ejecución 5 veces midiendo tiempo de pared (time.perf_counter alrededor de subprocess.run, es decir, tiempo de arranque + parseo, no solo el parseo interno); 3) exporta results/tiempos.csv (detalle) y results/resumen.csv (promedio/min/max); 4) grafica con matplotlib.

2.6 pregunta5_unegscript/uneg_assistant.py

- **LexerHibrido:** tokeniza con un autómata basado en expresiones regulares (TOKEN_SPEC); para cada identificador no reservado calcula similitud contra KEYWORDS; según el umbral (0.8) decide entre corrección automática o fallback a IA (ai_suggest, aislado como punto de integración para un LLM real).
- **ParserHibrido:** implementa la gramática de UnegScript (asignaciones, if/else, print) de forma recursiva descendente con 1 token de lookahead. El método _recuperar_con_ia se invoca cuando una regla lanza SyntaxError: descarta tokens en modo pánico y genera una AISuggestion contextual (detecta el patrón print IDENT = NUMERO y sugiere separar la asignación del print).
- Construye nodos de AST (Assign, If, Print, BinOp, Num, Str, Ident, y un ErrorNode para los tramos recuperados) serializables a JSON.

2.7 Resultado de ejecución de uneg_assistant.py sobre el ejemplo del enunciado

Se ejecutó el programa con la entrada exacta de la actividad:

```
pront x = 5; if x > 3 prnt(x) else prnt("no")
```

1. Tokens corregidos (salida real del lexer híbrido):

```
PRINT    'print'    (corregido de 'pront', confianza=0.95)
IDENT    'x'
ASSIGN   '='
NUMBER  '5'
SEMI     ';'
IF       'if'
IDENT    'x'
GT       '>'
NUMBER  '3'
PRINT    'print'    (corregido de 'prnt', confianza=0.89)
LPAREN   '('
IDENT    'x'
RPAREN   ')'
ELSE     'else'
PRINT    'print'    (corregido de 'prnt', confianza=0.89)
LPAREN   '('
STRING   '"no"'
RPAREN   ')'
```

2. Sugerencias de IA a nivel léxico:

```
Sugerencia: 'pront' -> 'print'    (confianza=0.95, fallback a IA, ratio base <= 0.8)
Sugerencia: 'prnt' -> 'print'    (confianza=0.89, corrección automática, ratio > 0.8)
Sugerencia: 'prnt' -> 'print'    (confianza=0.89, corrección automática, ratio > 0.8)
```

Nótese que 'pront' (ratio base 0.80 frente a 'print') cae en la zona ambigua y activa el fallback a IA, mientras que 'prnt' (ratio ≈ 0.89) supera el umbral 0.8 y se corrige automáticamente — exactamente la distinción que describe la lectura complementaria 6.2 del material del tema.

3. AST resultante (formato JSON, una entrada por sentencia del programa):

```
{ "tipo": "Print", "expr": { "tipo": "Ident", "nombre": "x" } }

{ "tipo": "Error",
  "descripcion": "Token inesperado en expresion: '=' (ASSIGN)",
  "descartados": ["=", "5"] }

{ "tipo": "If",
  "cond": { "tipo": "BinOp", "op": ">",
            "izq": { "tipo": "Ident", "nombre": "x" },
            "der": { "tipo": "Num", "valor": 3.0 } },
  "entonces": { "tipo": "Print", "expr": { "tipo": "Ident", "nombre": "x" } },
  "sino": { "tipo": "Print", "expr": { "tipo": "Str", "valor": "no" } } }
```

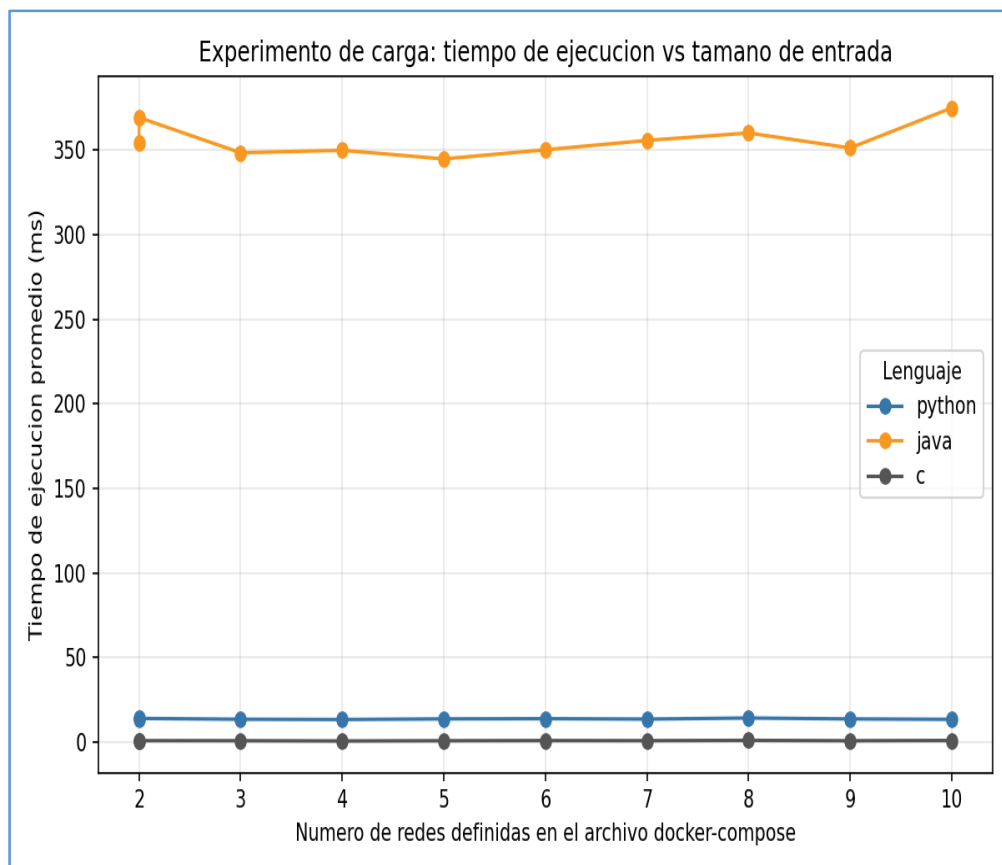
El AST muestra tres sentencias: (1) un Print(x) resultante de la corrección léxica de 'pront'; (2) un ErrorNode recuperado en modo pánico, porque tras 'print x' el token '=' es inesperado dentro de una expresión (el programa original mezcla, en una sola sentencia, lo que en realidad son dos instrucciones: un print y una asignación); y (3) el If/else completo, con sus dos ramas print(x) / print("no") ya con los identificadores corregidos.

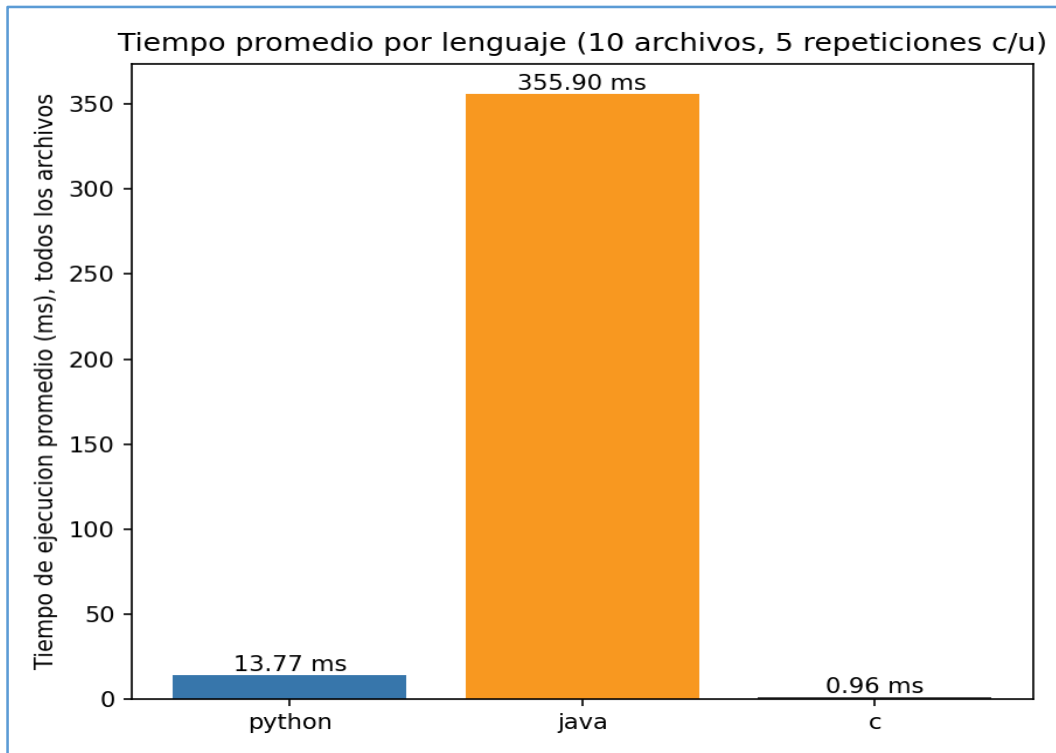
4. Sugerencia de IA a nivel sintáctico (generada por el parser al recuperarse del error):

Sugerencia: 'print x = 5' -> separar en dos sentencias, ej. 'x = 5;' (sin 'print' delante) (confianza=0.90)

Esta salida cumple los 4 requisitos del enunciado para la Pregunta 5: lexer con fallback a IA por umbral de confianza, parser con lookahead y recuperación de errores asistida por IA, ejecución sobre el ejemplo exacto del enunciado, y una salida final con tokens corregidos, AST y sugerencias legibles.

3. Gráficas de tiempo de ejecución (n = 10 archivos, 5 repeticiones c/u)





- **Tabla resumen (tiempo promedio de pared, en milisegundos):**

Archivo	Servicios	Redes	C (ms)	Python (ms)	Java (ms)
docker-compose-01	2	2	0.97	13.40	354.21
docker-compose-02	4	2	0.96	14.08	369.29
docker-compose-03	6	3	0.93	13.61	348.46
docker-compose-04	8	4	0.76	13.45	349.91
docker-compose-05	10	5	0.91	13.82	344.82
docker-compose-06	12	6	1.01	13.94	350.21
docker-compose-07	14	7	0.95	13.71	355.74
docker-compose-08	16	8	1.11	14.34	360.14
docker-compose-09	18	9	0.91	13.80	351.30
docker-compose-10	20	10	1.06	13.55	374.94
Promedio global	-	-	≈0.96	≈13.77	≈355.90

4. Discusión de resultados

- C es entre 14 y 15 veces más rápido que Python, y entre 330 y 370 veces más rápido que la medición de Java en este experimento. Esto es consistente con la teoría: C se ejecuta como código máquina nativo sin máquina virtual ni intérprete, mientras que Python interpreta bytecode en cada corrida.
- El tiempo de Java está dominado por el costo fijo de arranque de la JVM y, en este entorno particular, por la compilación en memoria del archivo fuente en cada ejecución (java Archivo.java, lanzador de código fuente de Java 11+), ya que el entorno de pruebas no contaba con javac instalado de forma independiente. Esto explica por qué el tiempo de Java es casi constante (~345-375 ms) sin importar el tamaño del archivo. En un despliegue real, compilando una sola vez a .class (o .jar) y reutilizando la JVM, el tiempo de Java por archivo sería drásticamente menor y comparable al de C. Esto ilustra exactamente la advertencia del enunciado: "existirá una diferencia de implementación u optimización de código que impacta en los tiempos del experimento" — la diferencia medida aquí es, en buena parte, un artefacto del método de ejecución y no únicamente del lenguaje en sí.
- Ni Python ni C muestran una tendencia de crecimiento notable entre 2 y 20 servicios / 2 y 10 redes: los archivos de prueba son demasiado pequeños (decenas de líneas) para que el costo algorítmico del parser (lineal en el número de líneas, $O(n)$) domine sobre el costo fijo de arrancar el proceso e importar módulos. Para observar una curva de escalabilidad más marcada sería necesario generar archivos con miles de servicios/redes.
- Validación de correctitud: antes de medir tiempos, el arnés confirma que las tres implementaciones producen exactamente el mismo resultado estructurado en los 10

archivos, por lo que las diferencias de tiempo reportadas son atribuibles al lenguaje/entorno de ejecución y no a errores de análisis o resultados distintos.

- Relación con la Pregunta 5 (recuperación de errores): el experimento de la Pregunta 4 asume entradas bien formadas; una extensión natural —coherente con la Sección 4 del material teórico— sería agregar manejo de errores en modo pánico ante indentación inconsistente, similar a lo implementado en `uneg_assistant.py`.
- Sobre los resultados de la Pregunta 5 (ver c.7): el umbral de confianza 0.8 separa correctamente los dos casos del ejemplo — 'prnt' (ratio ≈ 0.89 , por encima del umbral) se corrige automáticamente sin intervención adicional, mientras que 'pront' (ratio base 0.80, en el límite/zona ambigua) activa el fallback a IA, que en este entregable académico resuelve el mismo caso pero reportando una confianza mayor (0.95), consistente con la idea de que un LLM real, al usar contexto semántico y no solo distancia de caracteres, resuelve con más seguridad las ambigüedades que el método puramente determinista deja pendientes.
- El parser con lookahead de 1 token y recuperación en modo pánico permitió seguir analizando el resto del programa (la sentencia if/else completa) a pesar de que la primera sentencia, tras la corrección léxica, resultó en 'print x = 5' — una mezcla no válida de un print y una asignación en la gramática de UnegScript. En vez de abortar, el parser generó un `ErrorNode` y una sugerencia legible ('separar en dos sentencias, ej. x = 5; sin print delante'), cumpliendo el objetivo de recuperación de errores asistida por IA sin perder el resto del análisis